

Informe de flujo PID con anti-windup a FPGA-in-the-Loop

HDL Coder y HDL Verifier para MATLAB/Simulink

Ejemplo de controlador PID con anti-windup

Documento técnico

Proyecto PID

Responsable: Autor de Referencia

30 de diciembre de 2025

Índice general

1. Introducción	1
2. Modelado del PID en MATLAB/Simulink	2
2.1. Parte proporcional	2
2.2. Parte integral con anti-windup.....	2
2.3. Parte derivativa	2
2.4. Suma final y saturacion	2
3. Generación de HDL con HDL Coder	7

Índice de figuras

2.1	Modelo PID con anti-windup en Simulink.	3
2.2	Configuracion de tipos de datos y saturacion en bloques aritmeticos.....	4
2.3	Subsystem del PID en Simulink.....	4
2.4	Lazo cerrado en Simulink con actuador y planta.....	5
2.5	Respuesta del control PID con anti-windup en Simulink.	6
3.1	Configuracion para generacion en punto flotante.	7
3.2	Comparacion de recursos entre punto fijo y punto flotante.	7
3.3	Subsystem creado para la generacion de HDL.	8
3.4	Bloque FIL generado para el PID.	9
3.5	Lazo cerrado con bloque FIL en co-simulacion.....	9
3.6	Resultados de co-simulacion del lazo cerrado.	10
3.7	Respuesta en co-simulacion con referencia 10.	10
3.8	Resultados con anti-windup reducido y referencia 10.	10

1. Introducción

En este ejemplo se ilustra el diseño e implementación de un controlador PID con anti-windup en MATLAB/Simulink y el flujo de conversión a HDL con HDL Coder. El diseño del PID considera todos sus parámetros como entradas editables desde fuera. Luego, usando HDL Verifier, se busca probar en FPGA el diseño del PID con anti-windup dentro de un lazo cerrado en Simulink para una planta genérica, analizando el comportamiento ante cambios de ajuste sin depender de software adicional.

Además, se documenta la estructura discreta del PID, su implementación en Simulink y los pasos mínimos para convertir el subsystem a HDL y validarlo en un esquema FPGA-in-the-Loop.

2. Modelado del PID en MATLAB/Simulink

En esta etapa se modela un PID con anti-windup en tiempo discreto. El error se define como $e[k]$ y la salida del controlador como $u[k]$. La estructura considera parte proporcional, integral con anti-windup y derivativa.

2.1. Parte proporcional

La parte proporcional se implementa en el bloque `Proporcional_gain`:

$$P[k] = K_p \cdot e[k]$$

Responde de forma inmediata al error actual y no tiene memoria.

2.2. Parte integral con anti-windup

La parte integral se implementa en el bloque `I_component` con los terminos de ganancia integral y anti-windup:

$$I[k] = I[k - 1] + (K_i e[k] + K_{aw}(u_{sat}[k] - u[k]))T_s$$

El termino de anti-windup reduce la acumulacion cuando la salida esta saturada.

2.3. Parte derivativa

La parte derivativa se implementa en el bloque `D_component`:

$$D[k] = K_d T_s (e[k] - e[k - 1])$$

Aporta amortiguamiento y anticipa cambios rapidos del error.

2.4. Suma final y saturacion

La salida del PID se obtiene como:

$$u[k] = P[k] + I[k] + D[k]$$

Luego se aplica la saturacion para obtener $u_{sat}[k]$ y se realimenta la diferencia $u_{sat}[k] - u[k]$ al integrador.

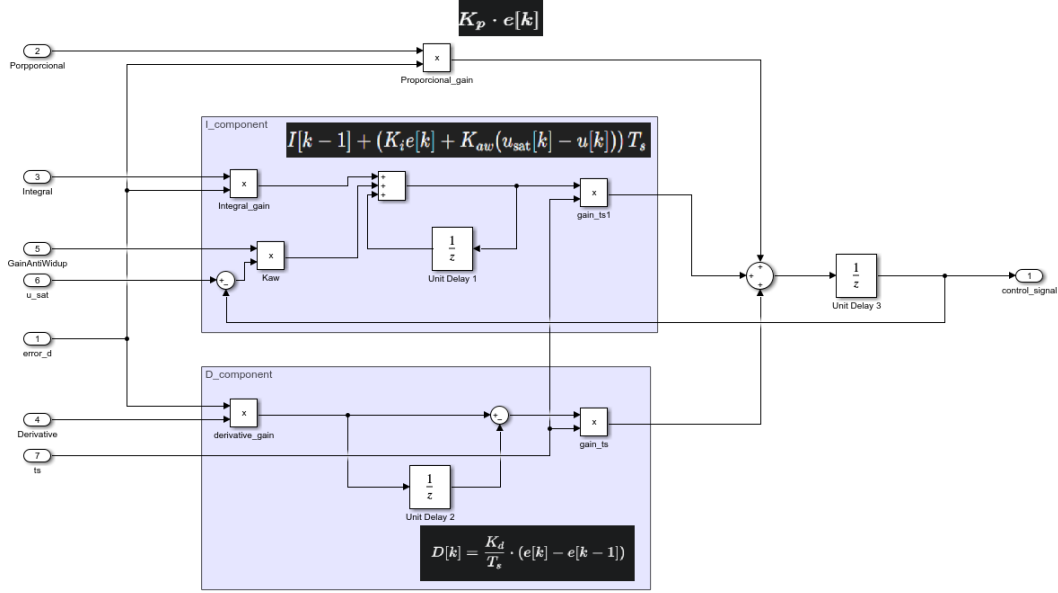
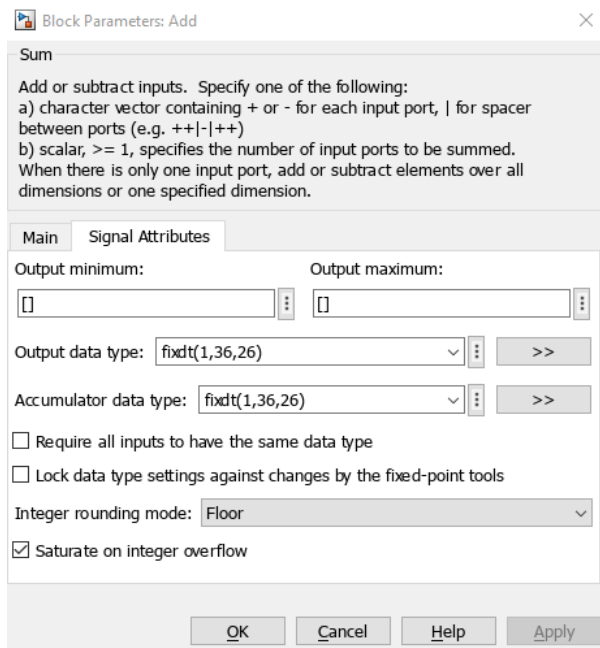


Figura 2.1: Modelo PID con anti-windup en Simulink.

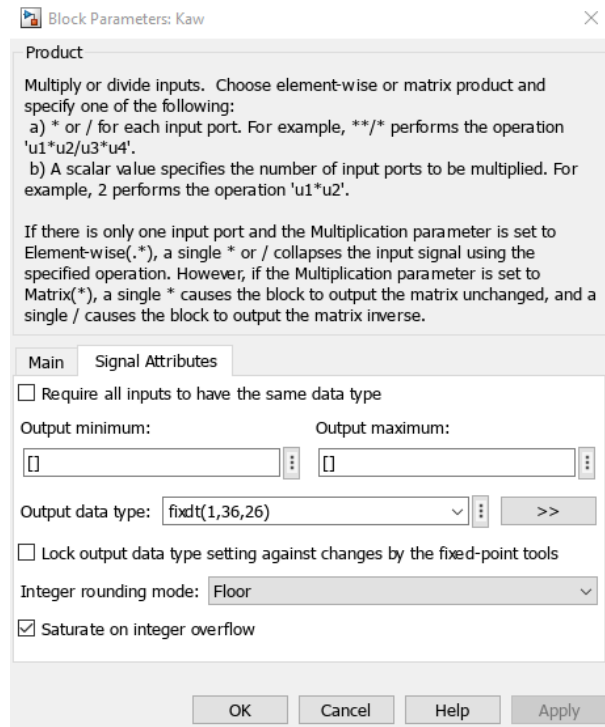
Para la implementacion en HDL, es necesario que todas las entradas y modulos trabajen con tipos de datos enteros o de punto fijo (`int`, `fixdt` o `logical`). En este caso se usara `fixdt(1,32,26)` para las entradas, y para los sumadores y multiplicadores se utiliza `fixdt(1,36,26)`. Esto significa que se usan $36 - 26 = 10$ bits para la parte entera y 26 bits de precision para la parte decimal, lo cual es adecuado porque el voltaje no suele tomar valores altos y se requiere mas detalle fraccional.

Ademas, se habilita la opcion **Saturate on overflow** para que, si el resultado supera el limite inferior o superior, quede limitado en ese valor y no ocurra un desbordamiento que invierta el signo o produzca saltos no deseados.

Tambien es necesario que, para usar HDL Coder, todos los bloques sean compatibles con el flujo de generacion. En particular, las sumas y multiplicaciones deben operar con tipos de datos enteros o de punto fijo, como `int`, `fixdt` o equivalentes, para evitar conversiones no soportadas.



(a) Configuración de tipos de datos y saturación en sumadores.



(b) Configuración de tipos de datos y saturación en multiplicadores.

Figura 2.2: Configuración de tipos de datos y saturación en bloques aritméticos.

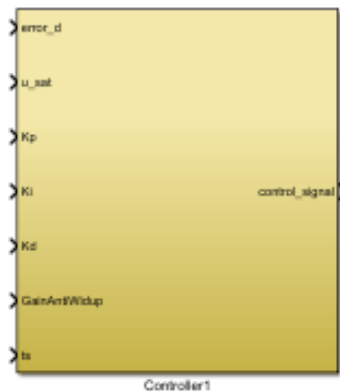


Figura 2.3: Subsystem del PID en Simulink.

teniendo el Subsystem del controlador PID pasamos a crearle un lazo cerrado para las pruebas.

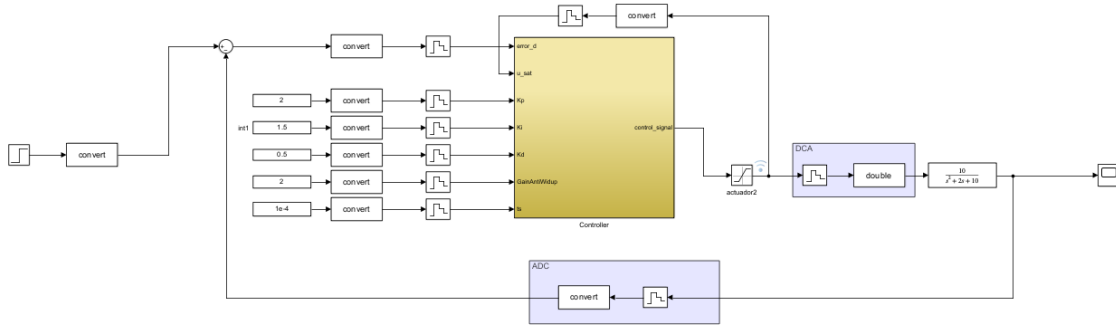


Figura 2.4: Lazo cerrado en Simulink con actuador y planta.

Para validar el controlador se diseña un lazo cerrado simple donde el actuador se modela con una saturación de ± 10 y la planta con la función de transferencia:

$$G(s) = \frac{10}{s^2 + 2s + 10}$$

Consideraciones principales:

- El PID diseñado es discreto, por lo que las entradas deben discretizarse usando un **Zero-Order Hold** el cual debe ser el mismo que T_s en este caso $1e-4$.
- Las entradas se manejan con `fixdt(1,32,26)`, por lo que se usa un bloque **Data Type Conversion** antes del controlador dado que simulink trabaja nativamente con `Double` en punto flotante.
- Entre el sistema discretizado y la planta continua se usa un **Zero-Order Hold** y un **Data Type Conversion** a `double`; esto simula un DAC.
- En la retroalimentación de la planta hacia el controlador se usa un **Zero-Order Hold** y un conversión de `double` a `fixdt(0,32,26)`; esto simula un ADC.
- Para la conexión de la salida del actuador con el controlador se usa un bloque **Conversion** y un **Zero-Order Hold**.

Para el controlador se usaron los parámetros $K_p = 2$, $K_i = 1.5$, $K_d = 0.5$, $K_{aw} = 2$ y $T_s = 1 \times 10^{-4}$. Con esta configuración se obtiene la siguiente respuesta en un lapso de 15 segundos, usando una señal de referencia de 2 que sube después del segundo 2.

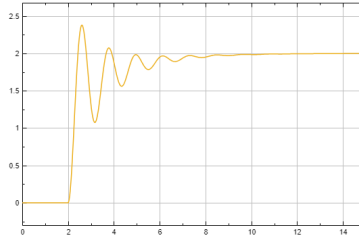


Figura 2.5: Respuesta del control PID con anti-windup en Simulink.

El siguiente paso es pasar el diseño creado en Simulink a HDL usando el flujo de HDL Coder.

3. Generación de HDL con HDL Coder

En Simulink se podría generar HDL usando punto flotante (los datos que MATLAB/Simulink emplean por defecto), pero esto consume muchos mas recursos en la FPGA y requiere mas ciclos para funcionar. En este ejemplo se utilizo punto fijo; aun asi, se adjunta la configuracion para punto flotante y una comparacion de utilizacion.

Para usar punto flotante se abre HDL Coder desde la seccion de Apps y se entra a la configuracion. En **Global settings** se debe colocar un **oversampling** mayor al valor por defecto (1), por ejemplo 100, aunque en este caso se requiere alrededor de 78. Ademas, en la configuracion se activa la casilla **Use floating point**.

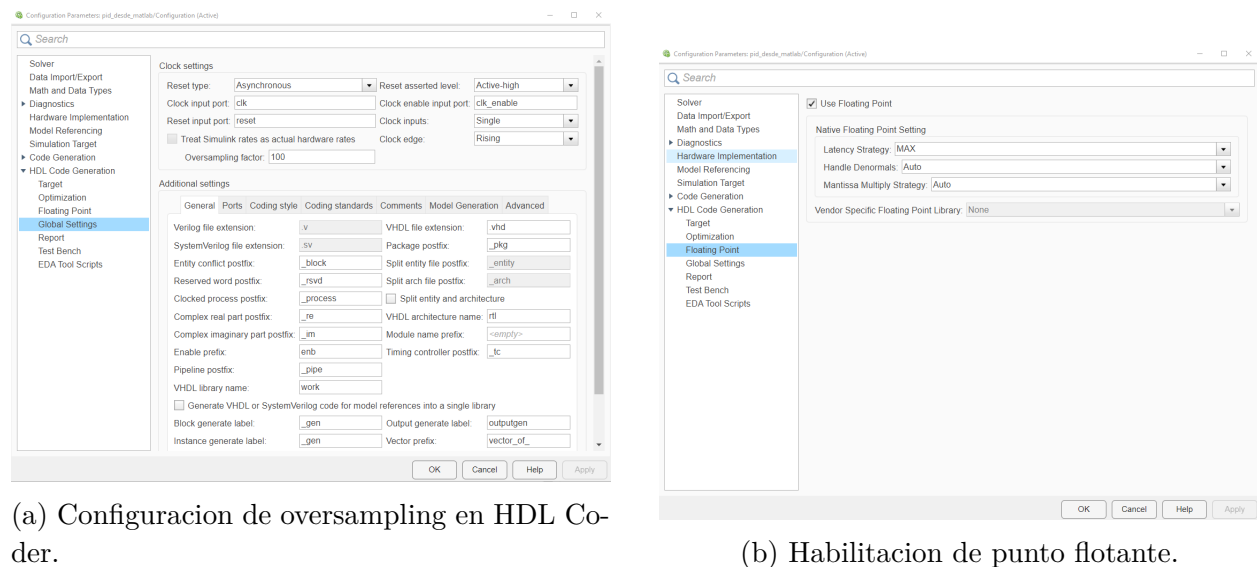


Figura 3.1: Configuración para generación en punto flotante.

Como ejemplo, se muestra la diferencia de utilización de recursos entre punto fijo y punto flotante.

Multipliers	6	Multipliers	6
Adders/Subtractors	7	Adders/Subtractors	79
Registers	7	Registers	949
Total 1-Bit Registers	144	Total 1-Bit Registers	18733
RAMs	0	RAMs	0
Multiplexers	5	Multiplexers	1007
I/O Bits	232	I/O Bits	516
Static Shift operators	0	Static Shift operators	6
Dynamic Shift operators	0	Dynamic Shift operators	12

(a) Utilización con punto fijo.

(b) Utilización con punto flotante.

Figura 3.2: Comparación de recursos entre punto fijo y punto flotante.

La utilización de recursos en punto flotante es mucho mayor que en punto fijo. Para generar

en punto flotante, todos los bloques deben trabajar con tipos **single** o **double**. En cambio, para que el diseño sea compatible con punto fijo, todos los bloques deben usar tipos enteros o de punto fijo, como **int**, **fixdt**, **fi** o **logical**.

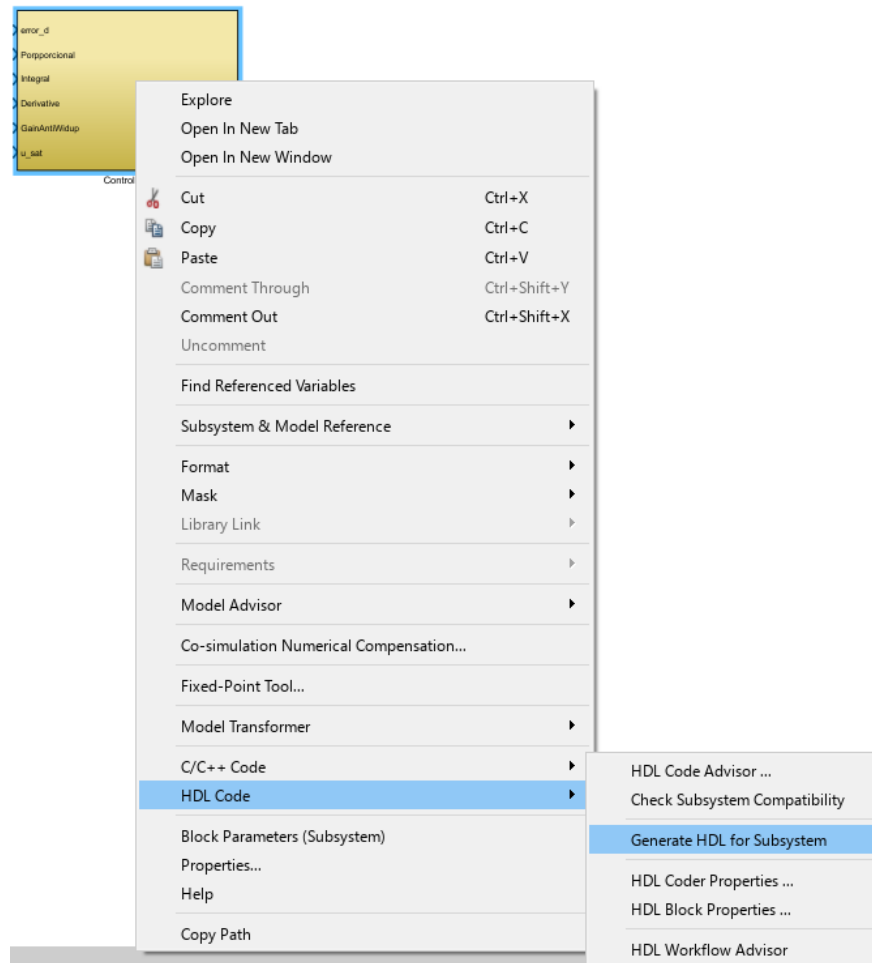


Figura 3.3: Subsystem creado para la generacion de HDL.

Este paso crea el archivo HDL en la carpeta del proyecto, en `hdlsrc/ProjectName/Nombre_Bloque.vhd` (o `.v` segun la opcion seleccionada).

Luego, usando el FIL Wizard con una configuracion estandar (sin ajustes avanzados), como se representa en la siguiente figura, se obtiene el bloque FIL para este PID.

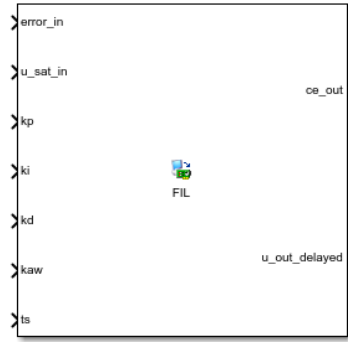


Figura 3.4: Bloque FIL generado para el PID.

Con el bloque FIL se replica el diagrama de lazo cerrado con los mismos parametros, y se ejecuta la co-simulacion para observar los resultados.

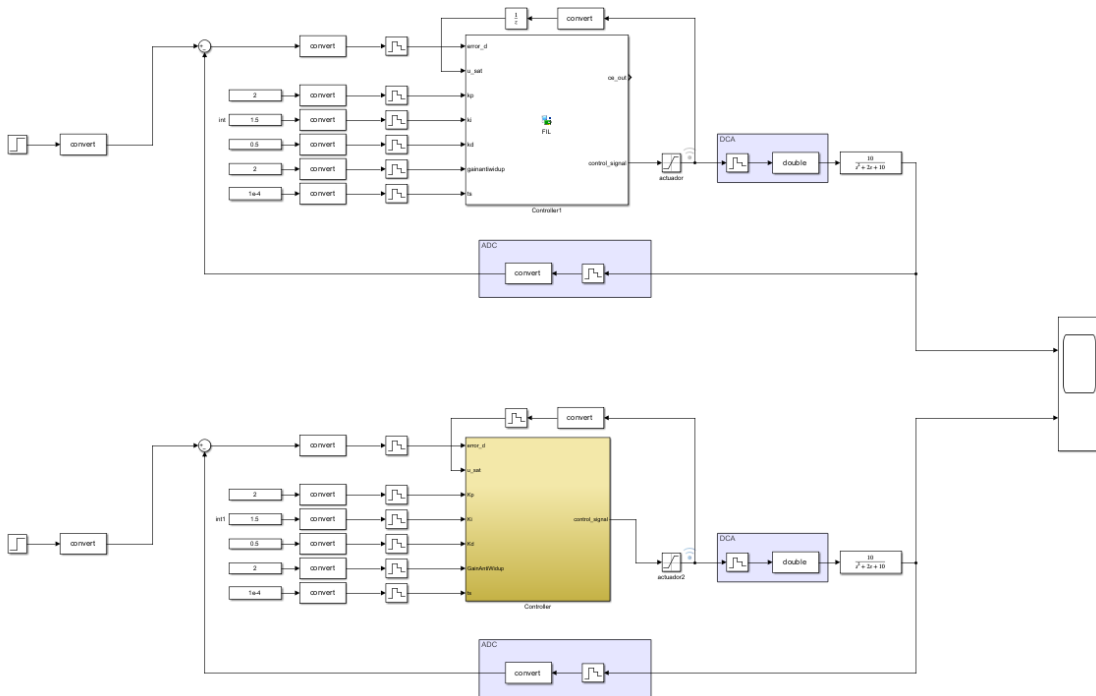


Figura 3.5: Lazo cerrado con bloque FIL en co-simulacion.

Se obtiene la salida de la planta y la salida del controlador, como se muestra a continuacion.

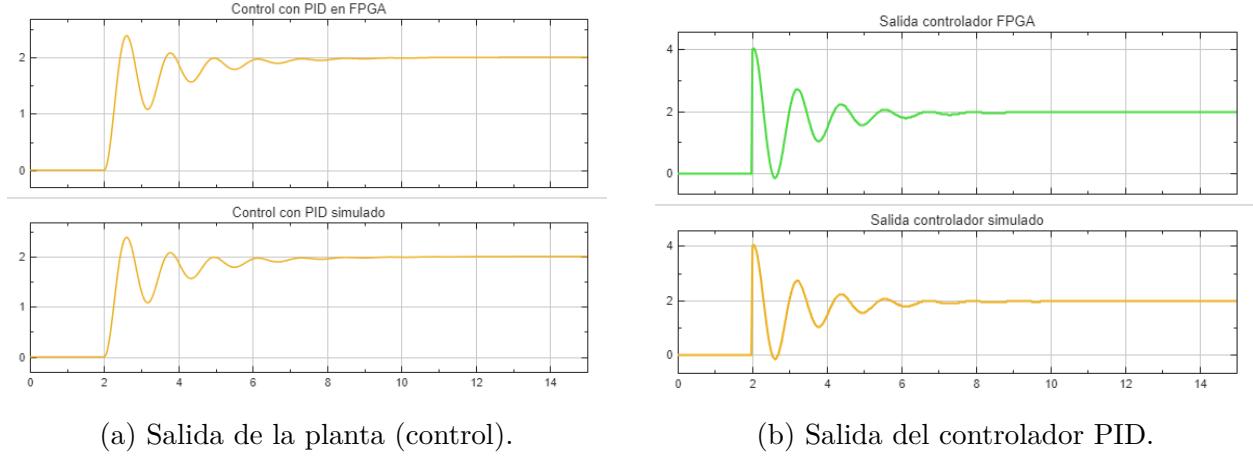


Figura 3.6: Resultados de co-simulacion del lazo cerrado.

Para demostrar la funcionalidad del anti-windup se realiza una prueba de seguimiento de referencia a 10 (la saturacion del controlador es de +10 a -10). Se obtienen los siguientes graficos:

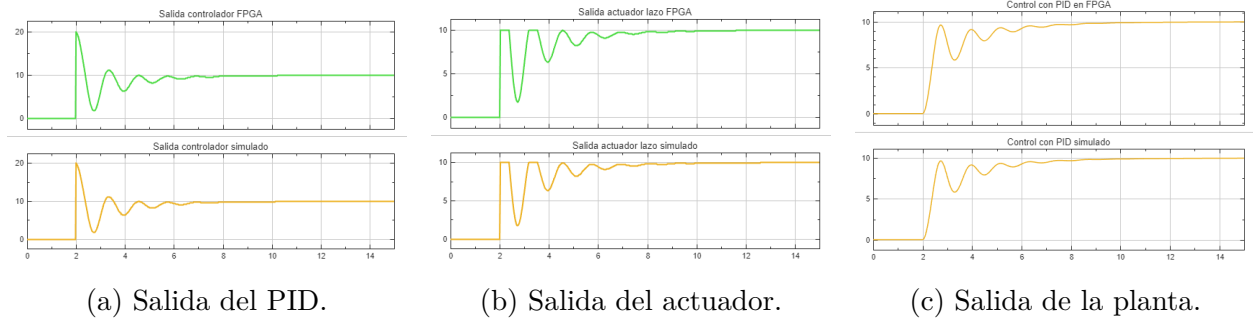


Figura 3.7: Respuesta en co-simulacion con referencia 10.

Luego se modifica la ganancia de anti-windup a 1 para que tenga un efecto minimo y se obtienen los siguientes resultados con referencia a 10:

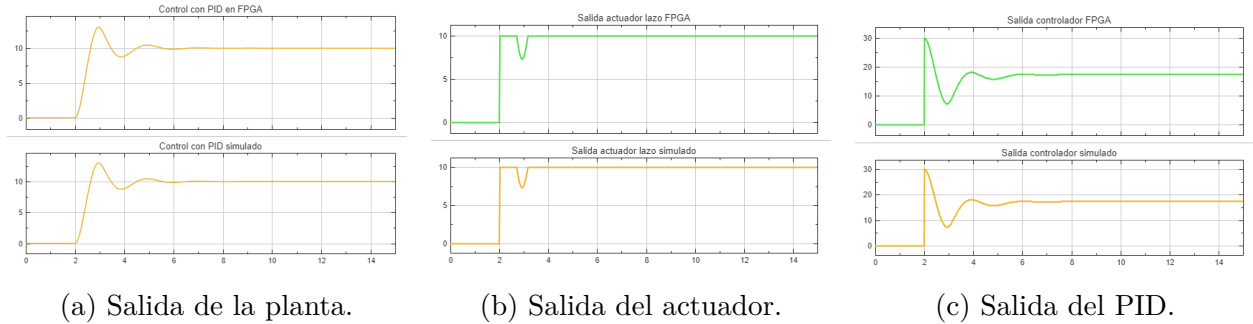


Figura 3.8: Resultados con anti-windup reducido y referencia 10.

En la salida de la planta se observa un sobreimpulso mayor que 10. En un sistema real, si el limite es 10 (por espacio, capacidad electrica, angulo, temperatura u otra restriccion), superar ese valor puede afectar gravemente el funcionamiento. En este caso el anti-windup cumple una funcion fundamental al reducir el sobreimpulso y evitar que se exceda el limite. Aumentar el valor del anti-windup disminuye aun mas el sobreimpulso, pero a cambio hace que la respuesta sea mas lenta.

Con esto terminado se ha podido verificar el funcionamiento en FPGA de un controlador PID con anti-windup, con todos sus parametros editables desde fuera, usando las herramientas MATLAB HDL Verifier y HDL Coder.